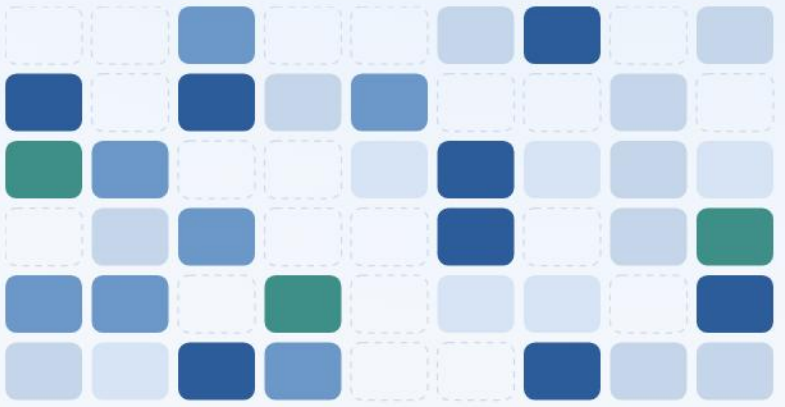
ПРЕЗЕНТАЦИЯ ПРОЕКТА · ВЕРСИЯ 1.3

NEX ×



шесть каналов × такты. Пунктир: пустая клетка, потерянная возможность

Планировщик широких команд для Эльбруса

Я сделал инструмент, который показывает, где код для процессора Эльбрус теряет такты, считает, как можно было бы лучше, и проверяет, помогает ли в этом нейросеть.

НАСТОЯЩИЙ КОД

947

блоков из 25 файлов lcc, оптимум каждого доказан

ЗАМЕРОВ

1 547

графов в трёх наборах, расхождений с эталоном 0

УЧЕБНЫХ СЦЕНАРИЕВ

17

у каждого один урок, например монополия делителя

О ПРОЕКТЕ · ЗАЧЕМ

Эльбрус делает много дел за такт, если их правильно разложить

ПРОЦЕССОР

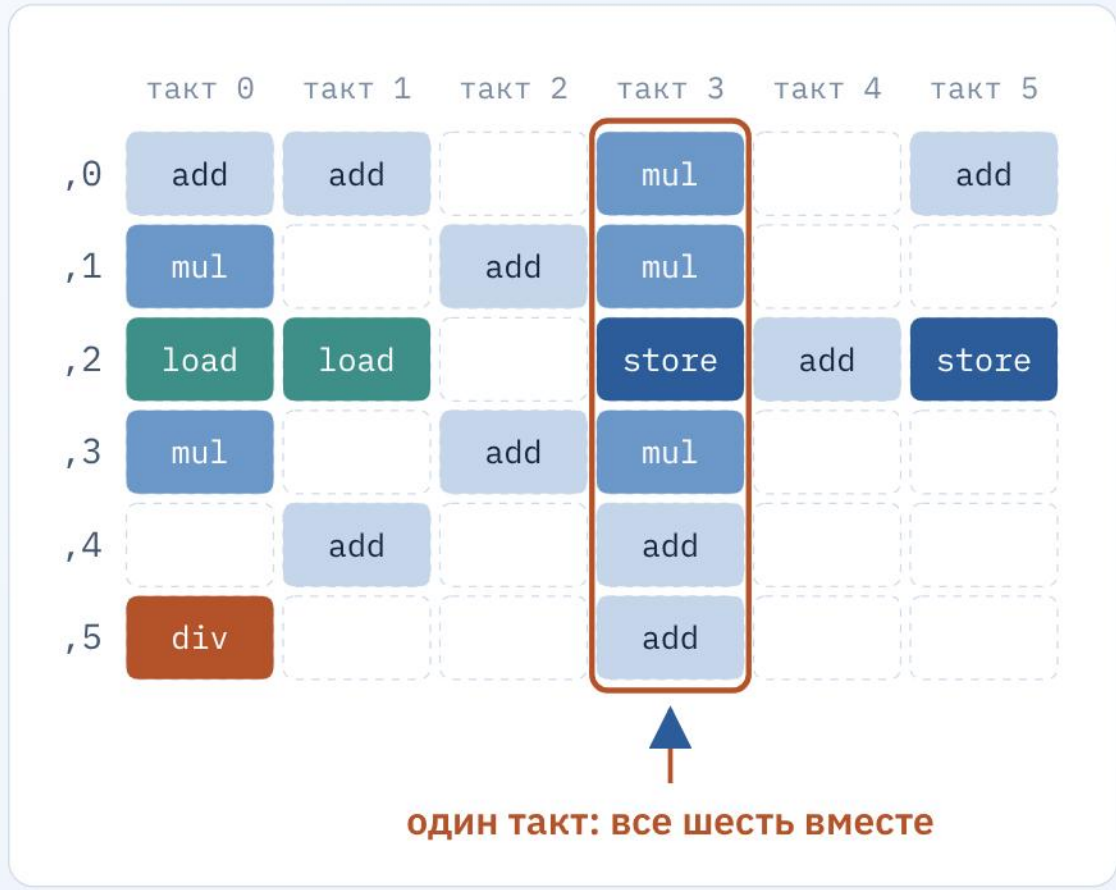
Эльбрус российский. За один такт он выполняет до **шести операций одновременно**: у него шесть каналов, как шесть касс.

ШИРОКАЯ КОМАНДА

Набор операций, которые запускаются вместе в одном такте. Это столбец на схеме.

КТО РАСКЛАДЫВАЕТ

Компилятор `lcc`, заранее. Процессор сам ничего не переставляет, поэтому плохая раскладка остаётся плохой.



6 каналов × 6 тактов, пунктир — пусто

ПРАВИЛО 1

Операция ждёт результат предыдущей: умножение 4 такта, загрузка 5, деление 11.

ПРАВИЛО 2

Не любая операция идёт в любой канал. Запись в память только в ,2 и ,5.

ПРАВИЛО 3

Делитель один: канал ,5, новое деление не чаще раза в два такта.

О ПРОЕКТЕ · ДЛЯ КОГО

Какую проблему решает NEX CLI и кому он нужен

ПРОБЛЕМА

Компилятор раскладывает операции быстро и по простому правилу: «сначала самое важное из готового». Проверить, что раскладка лучшая из возможных, он не может.

Остаётся вопрос без ответа: **сколько тактов теряется и можно ли их вернуть?**

ЧТО ДЕЛАЕТ NEX CLI

Берёт настоящий `.s`-файл от `lcc`, считает потолок (лучшее возможное расписание) и показывает разницу: где пустые клетки и почему они пустые.

Заодно проверяет, может ли нейросеть помочь с раскладкой.

ДЛЯ КОГО

Тем, кто пишет код под Эльбрус

Открыть вывод компилятора, увидеть пустые клетки и причину каждой. Любой ответ можно перепроверить командой.

Тем, кто делает компиляторы

Готовая линейка для сравнения: жадный, TINY AI и точный потолок с доказательством.

Тем, кто изучает ИИ в системном ПО

Честный стенд «поможет ли нейросеть». Результаты в репозитории, и те, где не помогла.

Проект бесплатный, лицензия MIT, работает на голом Python 3.10+ без обязательных зависимостей.

О ПРОЕКТЕ · КАК УСТРОЕНО

От файла компилятора до ответа «где теряем такты»

ШАГ 1

Файл от lcc

```
! <0001>
{
  ldw,0 [ a ], %g16
}
! <0002>
{
  ldw,0 [ b ], %g17
}
! <0003>
{
```

Настоящий ассемблер Эльбруса. В замерах 25 файлов.

ШАГ 2

Базовые блоки

блок 1 · 14 оп.

переход

блок 2 · 31 оп.

переход

блок 3 · 9 оп.

переход

блок 4 · 22 оп.

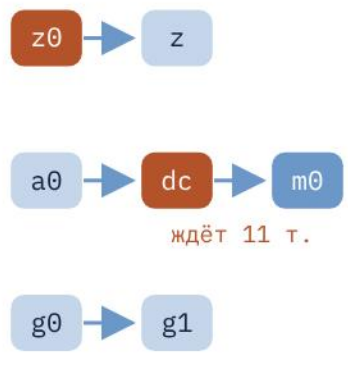
переход

блок 5 · 12 оп.

Код режется на куски без переходов внутри. Всего 947 блоков.

ШАГ 3

Граф зависимостей



Операции как узлы, стрелки как «после чего». В среднем 26 операций на блок.

ШАГ 4

Расчёт



Три планировщика считают расписание на примере slotclash, по одной модели машины.

ШАГ 5

Ответ

−1 такт из 24

Порт ,5 занят менее срочной z0.

Выдать dc раньше z0: z0 можно посчитать позже.

Сколько тактов потеряно и какая операция мешает.

ЕДИНСТВЕННАЯ МЕТРИКА

Число тактов на одном и том же графе и одной и той же модели машины. Больше ничего не сравнивается.

ВАЖНО

Это такты модели, а не замеры на железе. Аппаратная конвейеризация циклов (loop_mode) пока не моделируется.

ПЛАНИРОВЩИКИ · ПРОСТЫМИ СЛОВАМИ

Три способа разложить операции по тактам

ЖАДНЫЙ

Как в компиляторах

Идёт по очереди и каждый раз берёт самую важную из готовых операций. Решения необратимы: придержать ресурс «на потом» не умеет.

0,4 мс

на блок

ГАРАНТИЯ

оптимум не гарантирует

TINY AI

Маленькая нейросеть

Три сети по 10 тысяч чисел подсказывают порядок, отсрочки и «придержать». Расписание всё равно строит обычный код, поэтому оно законно по построению.

0,6 мс

на блок

ГАРАНТИЯ

не хуже жадного: ничья отдаётся жадному

ОРАКУЛ

Точный поиск

Перебирает варианты с отсечением и доказывает, что быстрее нельзя. Это потолок, с которым сравниваю остальных.

до 20 с

бюджет на граф

ГАРАНТИЯ

оптимум доказан, если уложился

ПО КАЧЕСТВУ жадный ≥ TINY AI ≥ оракул по тактам, чем меньше, тем лучше

ПЛАНИРОВЩИКИ · ПРИМЕР

Как теряется один такт: сценарий slotclash

ЖАДНЫЙ ТЕРЯЕТ ТАКТ

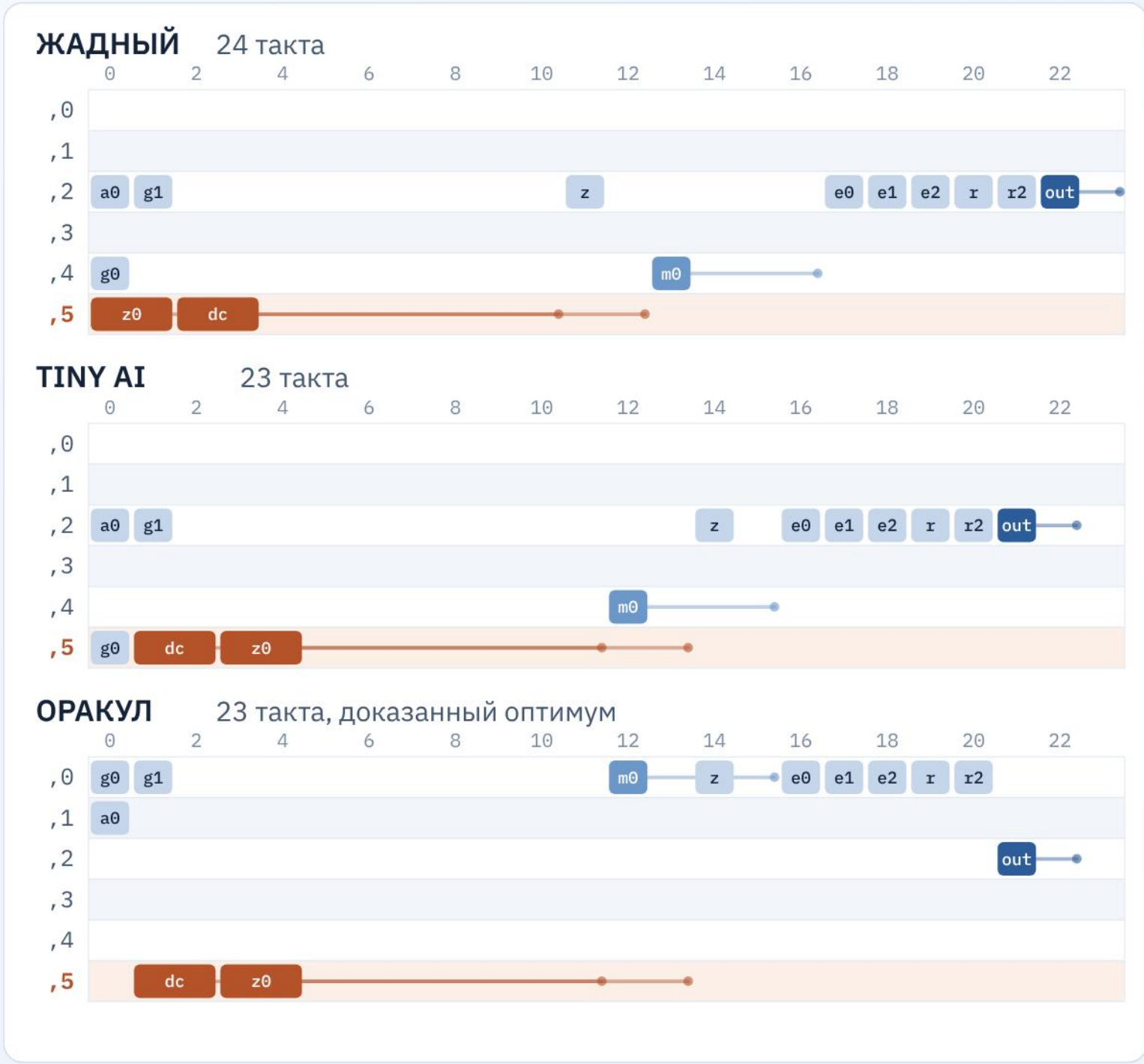
В такте 0 делитель ,5 свободен, и жадный ставит на него готовое деление z0. Но z0 никому не нужно срочно. Критичное dc готово в такте 1, а делитель занят: dc стартует в такте 2.

TINY AI НАХОДИТ ВЫХОД

Нейросеть и правило «придержать делитель» дают те же 23 такта, что и оракул. На это уходит миллисекунды, а не поиск.

ОРАКУЛ ДОКАЗЫВАЕТ

Перебор подтверждает: быстрее 23 тактов нельзя. Цепочка dc → m0 стартует на такт раньше, потому что z0 отложено.



сценарий slotclash, 13 операций. ■ деление ■ умножение ■ сложение ■ запись. Тонкая линия: результат в пути

TINY AI · УСТРОЙСТВО

TINY AI: маленькая сеть подсказывает, расписание строит код

ВХОД

Граф блока

Операции и зависимости

→

ШАГ 1

Признаки

17 чисел на операцию: тип, длина оставшегося пути, латентность

→

ШАГ 2

Три сети

Оценивают срочность, «не раньше» и «придержать»

→

ШАГ 3

Обычный код

Строит расписание по подсказкам

→

ШАГ 4

Лучший из 9

Берётся самое короткое. Ничья идёт жадному

ЧТО ЗНАЧИТ «ПОДСКАЗКА»

Сеть ничего не пишет. Она ставит каждой операции оценку срочности. Чем выше, тем раньше операцию надо ставить. Дальше работает обычный планировщик.



схема, значения условные

РАЗМЕР И СКОРОСТЬ

~32 тыс. чисел · 0,6 мс

во всех трёх сетях вместе, на блок. У Qwen2.5-3B три миллиарда чисел. Обучение: минуты на обычном процессоре

ГАРАНТИЯ

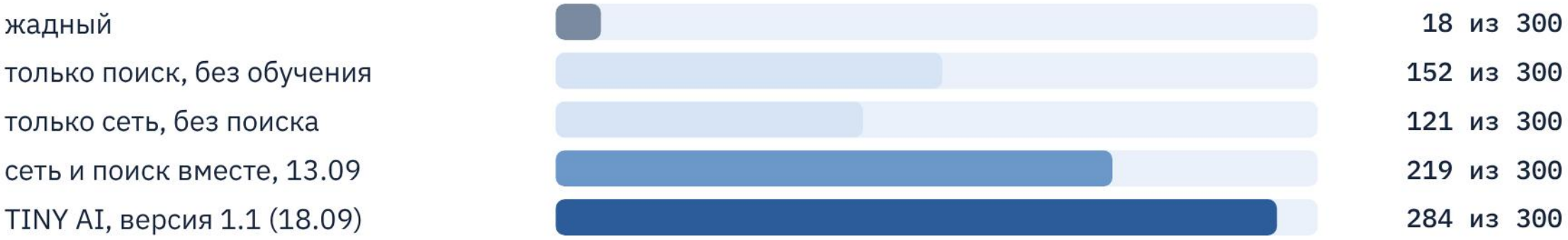
0 ошибок

незаконных расписаний и поражений жадному на 1 547 графах. Текст сеть не пишет

TINY AI · РЕЗУЛЬТАТ

TINY AI доводит 284 из 300 трудных графов до оптимума

Набор из 300 специально трудных графов, у которых оптимум известен. Считаю, сколько графов планировщик довёл до этого оптимума.



строки про отдельные части сняты на версии от 13.09

ЧТО ВИДНО

Ни поиск без обучения, ни обучение без поиска не дают много. Вместе они дают почти всё.

ЧТО ДОБАВИЛ 18.09

Ещё три кандидата: «придержать» (hold), ручные держки монопольного порта и веер по держкам.

ОСТАТОК

16 графов из 300. Все не дотянули ровно на один такт.

ЗАМЕРЫ · НАСТОЯЩИЙ КОД

На настоящем коде запас небольшой, и это главный итог

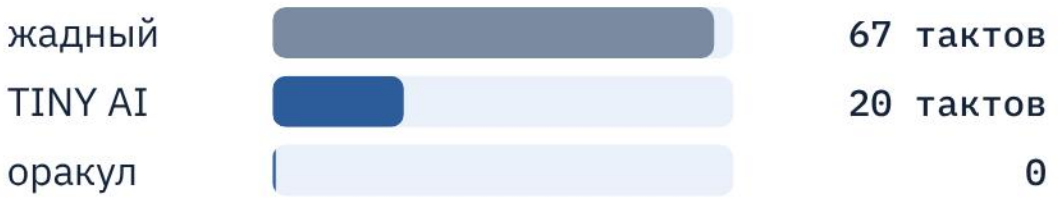
ЧТО ЗА ТЕСТ

Настоящий компилятор lcc выдал 25 файлов. Я разрезал их на 947 блоков. Для каждого блока оракул доказал минимум тактов. Теперь смотрим, насколько к нему близки жадный и TINY AI.

БЛОКОВ С ОПТИМАЛЬНЫМ РАСПИСАНИЕМ, ИЗ 947



ЛИШНИХ ТАКТОВ НА ВСЕ 947 БЛОКОВ



всего у оптимума 13 131 такт; у жадного 13 198, у TINY AI 13 151

ПОТЕРИ ЖАДНОГО

0,51%

от всех тактов настоящего кода

ПОТЕРИ TINY AI

0,15%

вернул 70% потерь жадного

БОЛЬШИЕ ГРАФЫ LCC

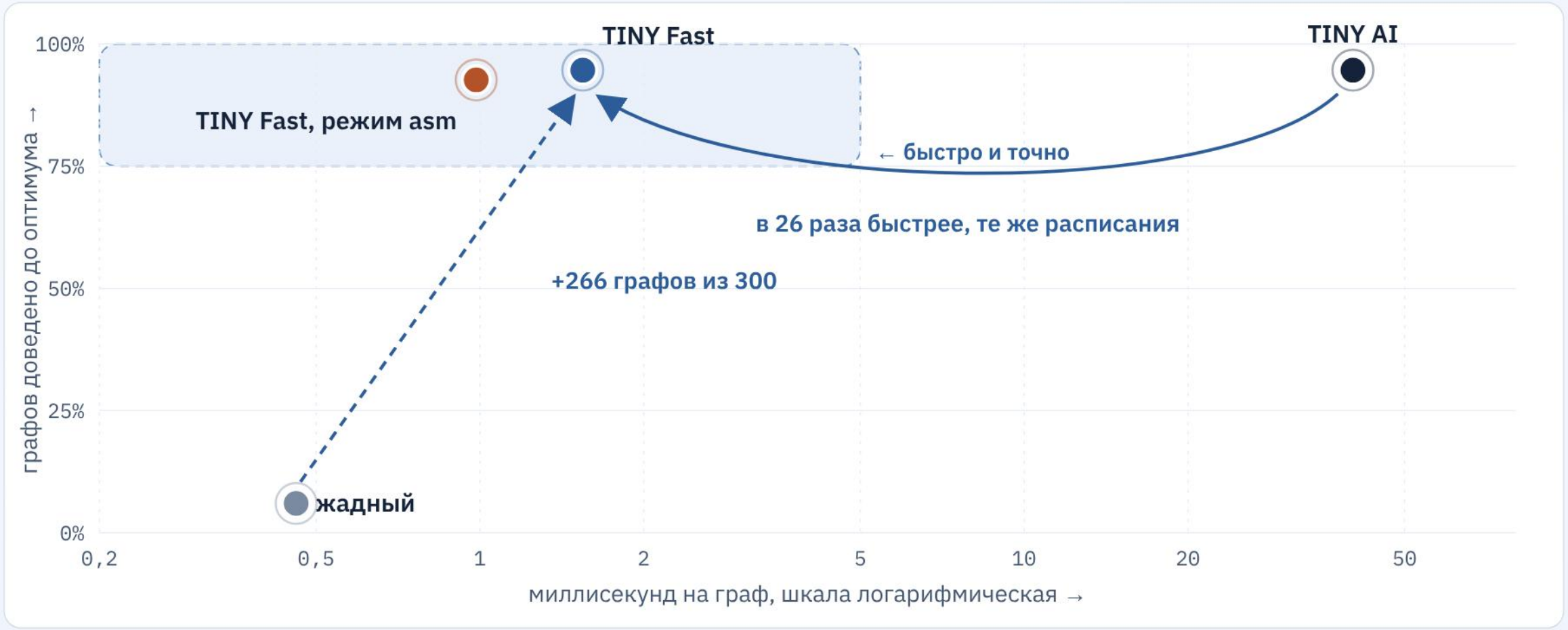
7 / 6 / 0

побед, ничьих, поражений на 13 графах по 112–386 операций

ЧЕСТНЫЙ ВЫВОД На настоящем коде жадный уже оптимален в 94% блоков. Запас есть, но маленький: доли процента тактов. TINY AI забирает большую его часть.

TINY AI · СКОРОСТЬ

TINY Fast быстрее TINY AI в 26 раз и даёт те же расписания



TINY AI

Первая версия: считает все кандидаты до конца. Надёжно, но 40 мс на граф.

TINY FAST

Новая версия того же TINY AI. Останавливается, когда достигнута нижняя граница тактов. Результат тот же, в 26 раз быстрее.

РЕЖИМ ASM

Ещё быстрее, но на 6 графах из 300 хуже. Гарантии нет, поэтому выключен по умолчанию.

ЗАМЕРЫ · СВОДКА

Сводная таблица: кто сколько графов довёл до оптимума

ПЛАНИРОВЩИК	НАСТОЯЩИЙ КОД ИЗ 947	ТРУДНЫЕ ИЗ 300	ШИРОКИЕ ИЗ 300	СЦЕНАРИИ ИЗ 17	МС НА БЛОК
Оракул точный поиск	947 100%	300 100%	300 100%	17 100%	до 20 с
TINY AI первая версия	927 98%	284 95%	216 72%	17 100%	44,7
TINY Fast новая версия, те же расписания	927 98%	284 95%	216 72%	17 100%	0,59
TINY Fast, asm без гарантии	927 98%	278 93%	216 72%	не замерено	0,21
Жадный как в компиляторах	890 94%	18 6%	216 72%	15 88%	0,40

оптимальных графов по наборам. Чем темнее клетка, тем ближе к максимуму. Мс на блок: набор настоящего кода, один процесс

НАСТОЯЩИЙ КОД
947 блоков из файлов lcc

ТРУДНЫЕ
синтетика, где жадный ошибается чаще всего

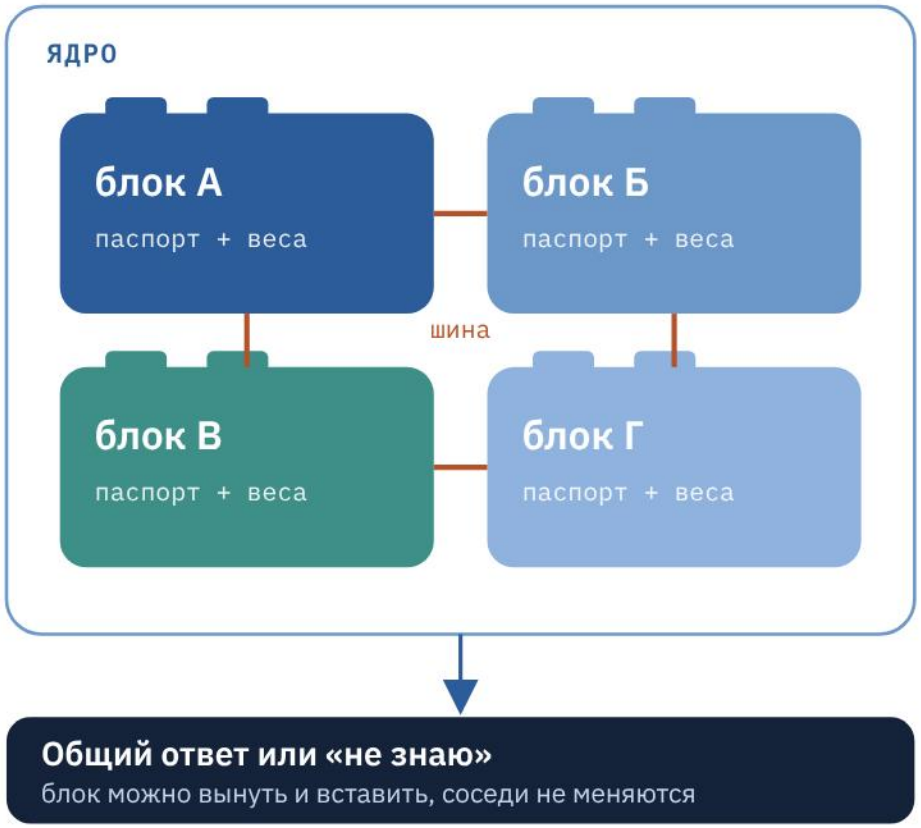
ШИРОКИЕ
много независимых операций, жадному хватает

СЦЕНАРИИ
17 учебных примеров с уроком в каждом

КАК ЧИТАТЬ На широких графах TINY AI не лучше жадного (216 из 300 у обоих): там нечего улучшать. На трудных и настоящих он забирает почти всё.

NNL · ЧТО ЭТО

NNL: отдельный проект, который я встраиваю в NEX CLI



ЧТО ТАКОЕ NNL

Neural Network Lego. Нейросеть, собранная из блоков, как из конструктора. Каждый блок — маленькая сеть со своим паспортом и файлом весов. Ядро собирает ответы блоков и умеет честно сказать «не знаю».

ОТКУДА ОН У МЕНЯ

NNL — другой продукт. Он пришёл из отдельного проекта, и я встраиваю его в NEX CLI поэтапно. Пока это сырая фоновая задача, а не активное ядро.

ЗАЧЕМ В NEX CLI

Проверить, умеет ли такой конструктор выбирать подходящий способ расчёта и оценивать, есть ли что улучшать.

РАНЬШЕ

Отдельный проект

NNL жил в своём репозитории

СЕЙЧАС

Фон в NEX CLI

Оценивает и подсказывает, но не строит расписание

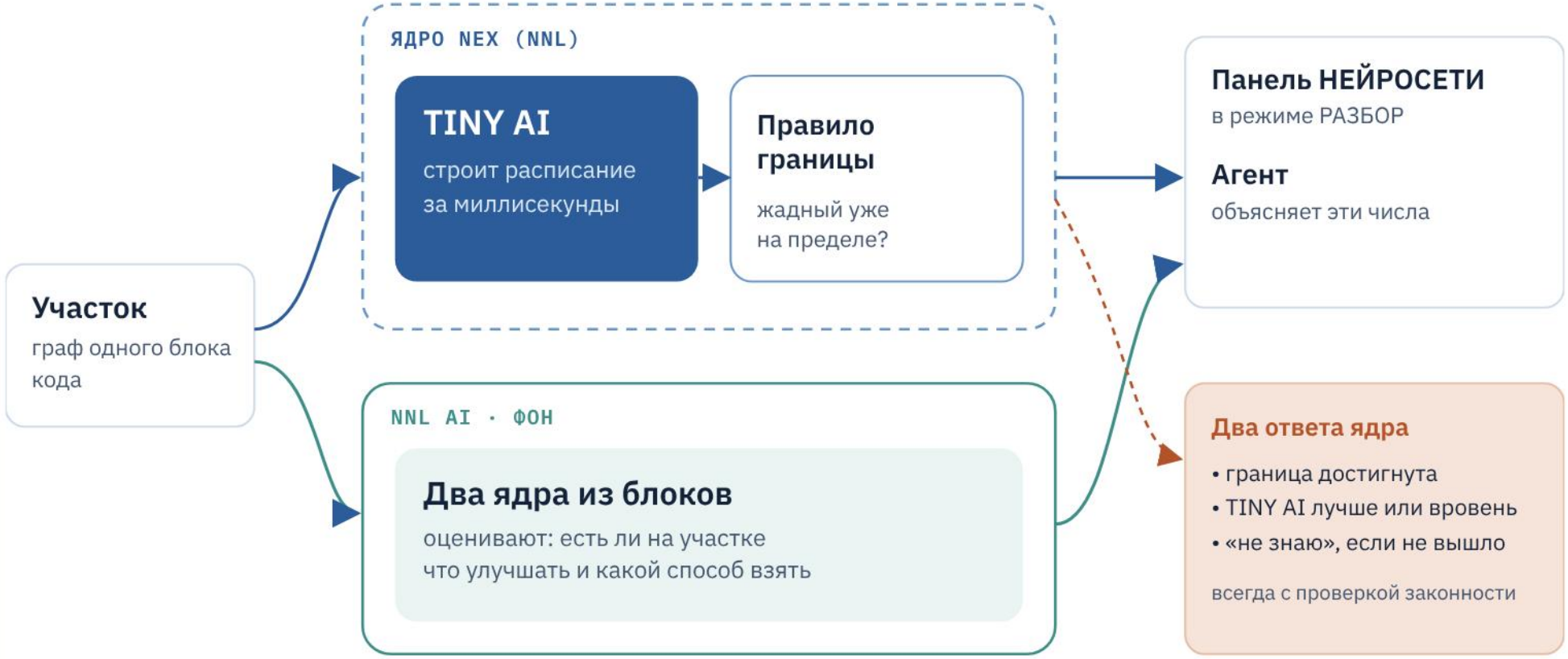
ДАЛЬШЕ

Расширять по результатам

Если окупится, станет активным ядром

NNL · АРХИТЕКТУРА

Две сети рядом: TINY AI строит, NNL AI оценивает



КАК ОНИ ДЕЛЯТ РАБОТУ

TINY AI считает расписание. NNL AI не соревнуется с ним: он работает рядом и оценивает, стоит ли вообще что-то улучшать.

СТАТУС

В ядре NEX пока один блок «планировщик», поэтому шина почти не работает: блоки ничего друг у друга не читают.

NNL · ЧТО ПОЛУЧИЛОСЬ

NNL выбирает способ расчёта: работает, но выигрыша пока нет

Задача NNL здесь не строить расписание, а выбирать, каким из способов TINY AI считать блок. Поэтому сравниваю его не с TINY AI, а с обычными классификаторами на тех же данных.



настоящий код, 43 трудных графа из 947. Полоса: разброс трёх запусков, точка: среднее

ПРОТИВ ОБЫЧНЫХ КЛАССИФИКАТОРОВ

Ядро А чуть лучше. Ядро Б вровень. Обоих вместе нет смысла включать: они не лучше случайного отказа (33 против 28).

УГАДАТЬ КЛАСС КОМАНДЫ

Ядро 0,84 против 0,93 у обычной логрегрессии. Принимает ответ редко (30 из 158), зато почти всегда верно (29).

НА НАСТОЯЩЕМ КОДЕ

Качество почти случайное: ядра обучены на синтетике и настоящий код не видели.

ИТОГ NNL технически работает, но выигрыша пока не даёт, поэтому остаётся фоном. Рабочая часть рядом: правило границы снимает 883 из 947 блоков без единой ошибки.

ЧТО НЕ ПОЛУЧИЛОСЬ

Языковая модель не справилась, и я заменил её на TINY AI

ЧТО Я ПРОБОВАЛ

Взял языковую модель Qwen2.5-3B и дообучил её (метод LoRA), чтобы она писала расписание текстом: строка за строкой, «такт, канал, операция».

Gemma и Phi я тоже проверял. Эффективности они не показали, даже с обученными параметрами.

ПОЧЕМУ НЕ ВЫШЛО

Модель «рассуждает» словами, а расписание — строгая задача с правилами. Одна ошибка в строке, и вся раскладка недействительна. К тому же долго.

СКОРОСТЬ

3–6,5 мин

на один граф, 1,5–6,5 токена в секунду

КОНТЕКСТ

4096

токенов: на графах от 300 операций ответ не помещается

ЗАКОННОСТЬ

61%

расписаний незаконны на широком наборе

НАСТОЯЩИЙ КОД

0 из 13

графов, где модель обошла случайный перебор

ЧТО ПОЛУЧИЛОСЬ ВЗАМЕН

3 млрд

чисел у Qwen, минуты на граф

30 тыс.

чисел у TINY AI, миллисекунды

ВТОРАЯ ОШИБКА: НЕВЕРНАЯ МОДЕЛЬ МАШИНЫ

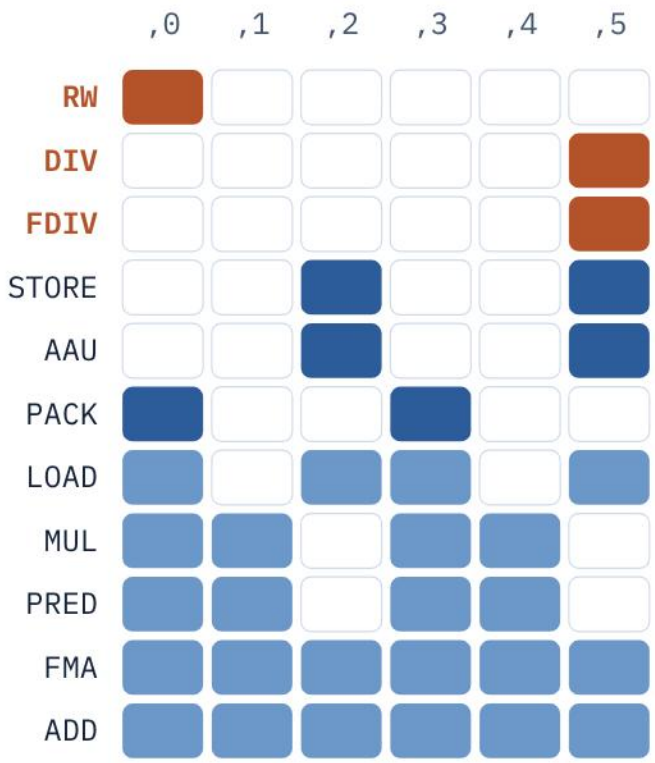
Две недели я считал по модели, где умножитель был один. «Разрыв 18%» оказался 3%. Ассемблер показал, что умножителей четыре.

МОДЕЛЬ МАШИНЫ

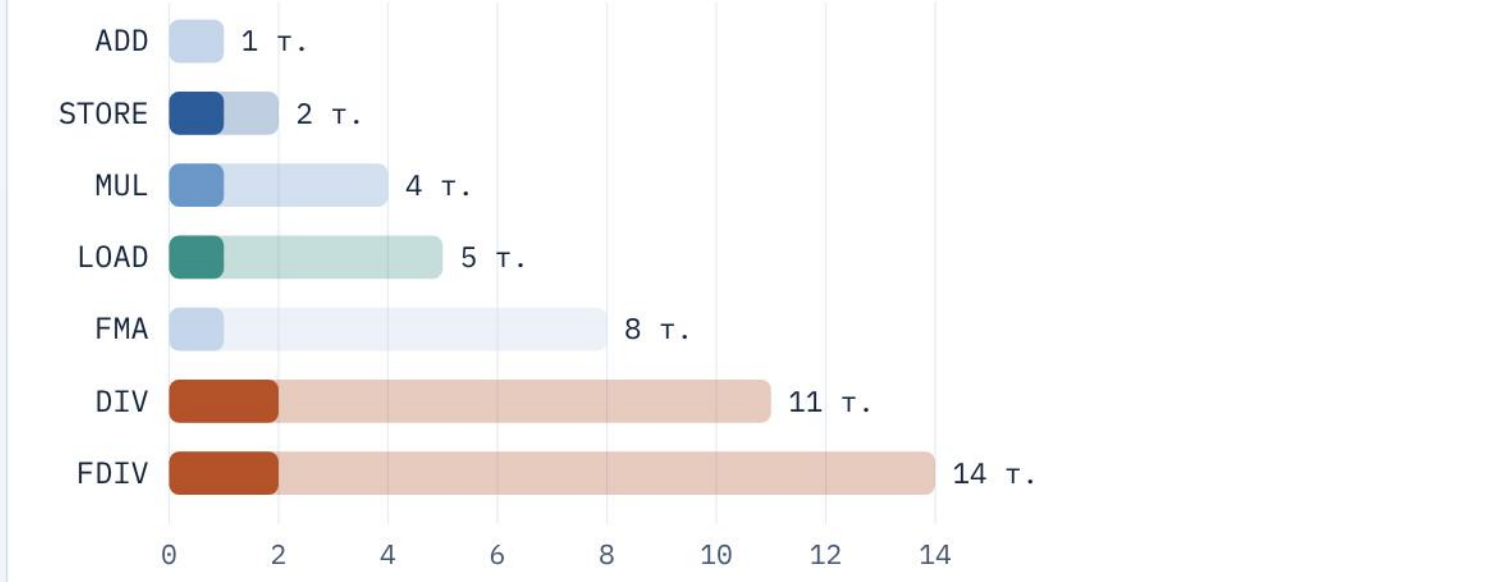
Модель Эльбруса e2k-v6: шесть каналов и один делитель

ЗАЧЕМ ОНА НУЖНА

Модель машины — это правила игры: в какие каналы какая операция может встать и сколько тактов ждать результат. Без неё нельзя посчитать такты, ни мои, ни чужие.



какая операция в каком канале.
Оранжевый: канал единственный



порт занят (тёмная часть) и результат готов (вся полоса)

ОТКУДА ЧИСЛА

С компилятора lcc и от ассемблера: он сам отказывается кодировать операцию не в том канале. На живом Эльбрусе числа не проверены, поэтому все такты в презентации модельные. Часть примеров .s в проекте я писал руками, настоящие выводы лежат в examples/probes.

ИНТЕРФЕЙС · КАРТА

Четыре режима: КОД, РАЗБОР, АГЕНТ и ЯДРО

КЛАВИША 1

КОД

РАБОЧЕЕ МЕСТО

Редактор .s, расписание, замечания и разбор вкладками. Сюда приношу свой файл.

КЛАВИША 2

РАЗБОР

ВЕСЬ ГРАФ

Решётка тактов × каналов и панели, отвечающие про клетку под курсором.

КЛАВИША 3

АГЕНТ

ДИАЛОГ И ШТАБ

Вопрос обычным языком. Ответ строится по уже посчитанным числам.

КЛАВИША 4

ЯДРО

ИНТЕРПРЕТАТОР

Считаю выражения, из них строится граф. Команда go отдаёт его в РАЗБОР.

общий участок и расписание

агент читает посчитанные числа

go: граф из ЯДРА открывается в РАЗБОРЕ

КАК ПОПАСТЬ

`python3 -m vliw`: экран выбора, цифры 1–4. Или сразу `python3 -m vliw run slotclash`.

НЕ ПУТАТЬ

Режим ЯДРО (интерпретатор) и ядро NEX (нейросеть NNL) — разные вещи с похожим названием.

КОД · УСТРОЙСТВО ЭКРАНА

КОД: рабочее место, куда я приношу свой файл



Файлы слева, редактор по центру, папки справа, нижняя панель для вывода. Три тумблера в шапке открывают и закрывают колонки и док.

КОД · ПРОГОНЫ

КОД в деле: прогон примеров

```
$ python3 -m vliw load examples/stress/deep/hard_mix.s
загружен · hard_mix.s
■ резерв 15 т. (48%)
lcc 31 т.  →  поиск 16 т.  →  предел 16 т.

35 операций · 30 широких команд
ADD×12 MUL×8 LOAD×4 DIV×2 FMUL×2 ...
```

```
$ python3 -m vliw load examples/code/broken.s
■ расписание компилятора не восстановлено
строка 18: канал ,5 в такте 1 уже занят:
  sdivs из строки 15 держит порт 2 т.
строка 21: muls в канале ,2 — такого канала
  у операции нет
```

ФАЙЛ	ЧЕМУ УЧИТ	КАК БЫЛО → ПОИСК
slots.s	восемь умножений идут по одному в такт, а каналов для них четыре	**15 → 8**
divport.s	два деления на единственный делитель	**22 → 18**
loads.s	четыре загрузки можно поднять выше	**10 → 7**
basic_pipeline.s	загрузка, умножения, свёртка сложением	**11 → 9**
hard_mix.s	делитель, восемь умножений, цепочка загрузок, плавающая точка	**31 → 16**
latency.s	цепочка зависимых умножений: быстрее нельзя	**12 → 12**

цифры — такты: как записано в файле и что нашёл точный поиск

ЧЕСТНО О ПРИМЕРАХ

Эти файлы я писал руками для проекта. Настоящий вывод lcc лежит в examples/probes и в 25 файлах real_candidates_100_400.

КОД · НИЖНЯЯ ПАНЕЛЬ

Шесть вкладок дока: от журнала до мини-агента

вывод | замечания | разбор | **расписание** | ядро | агент

F11: текущая вкладка на весь низ экрана

F12: показать или спрятать весь док целиком

ВКЛАДКА

вывод

Журнал команд и прогонов, как в консоли.

ВКЛАДКА

замечания

Находки линтера с номерами строк. Клик ведёт курсор к строке.

ВКЛАДКА

разбор

Полный разбор строки под курсором: кто кормит операцию и что с ней сделает поиск.

ВКЛАДКА

расписание

Решётка тактов × каналов для текущего буфера.

ВКЛАДКА

ядро

Мини-версия режима ЯДРО: считаю выражения, не выходя из КОДа.

ВКЛАДКА

агент

Мини-диалог: свой вопрос и список того, что видит агент.

`F5 прогнать` `F6 переписать по оракулу` `F2 переименовать` `^S сохранить` `^G объяснить строку` `^P или / команды` `Esc шаг назад`

ОГОВОРКА ПРО F6 F6 переписывает буфер по расписанию оракула и единственное в проекте пишет код. После него смотрю вкладку «замечания»: редкие случаи ломают имена регистров.

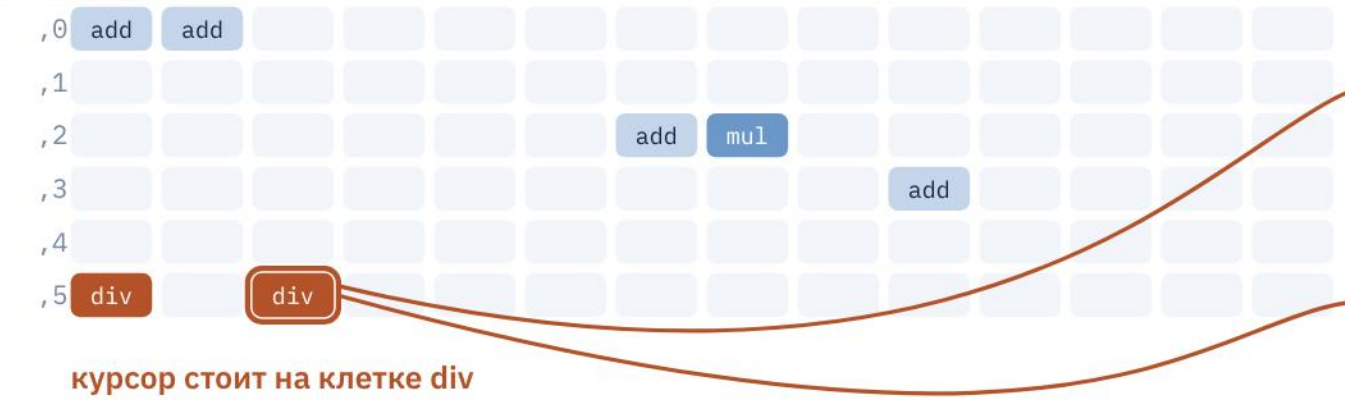
РАЗБОР · УСТРОЙСТВО ЭКРАНА

РАЗБОР: один курсор, семь панелей

УЧАСТОК

slotclash | divpressure | memstream | ptrchase | wide_ilp | ifconv

РАСПИСАНИЕ



ЧИСЛА

baseline 24 · оракул 23 · выигрыш −1

ПОЧЕМУ ЗДЕСЬ

отвечает про клетку под курсором, даже пустую

ДИАГНОЗ

находки по цене: порт ,5 занят менее срочной z0

GIT

журнал прогонов: что вводил, что вышло

МАШИНА

матрица портов и латентности

ГЛАВНОЕ, ЧТО НАДО ПОНЯТЬ

Решётка и правая колонка читают один и тот же курсор. Где курсор, там и ответ всех панелей.

УПРАВЛЕНИЕ

`F2` фокус на решётку · `↑↓←→` курсор · `Enter` разбор такта · `v` жадный/оракул · `n` следующая находка

РАЗБОР · ПАНЕЛИ

Двойной клик: каждая панель раскрывается на весь экран

Двойной клик по любой панели раскрывает её на весь экран. Там больше функций. `Esc` возвращает обратно.

ПАНЕЛЬ	КАК ОТКРЫТА СРАЗУ	ПОСЛЕ ДВОЙНОГО КЛИКА ⤢
РАСПИСАНИЕ	решётка тактов × каналов, простои схлопнуты в одну строку	шире колонки (w), все такты (x), только соседи (z), критический путь (p), вид жадный / оракул / модель (v)
ЧИСЛА	baseline, оракул, предел, выигрыш в тактах и процентах	откуда взялся предел: цепочка критического пути и ресурсное ограничение по отдельности
ПОЧЕМУ ЗДЕСЬ	ответ про клетку под курсором, даже пустую	кнопки «операция», «по цепочке», шаги по критическому пути
ДИАГНОЗ	лучшие находки доктора, отсортированы по цене	фильтр: все, критично, средне, мелко, предел. Полный текст каждой: почему и что делать
МАШИНА	какая операция в каком канале живёт	сортировка по числу операций или латентности, фильтр по имени операции
GIT	журнал прогонов этой сессии (не git-репозиторий)	полный журнал, ↻ повторить последний прогон, очистить

СПРАВОЧНИК ИИ

`Tab` открывает справочник в любой момент, в том числе после раскрытия панели.

17 СЦЕНАРИЕВ

Панель УЧАСТОК: каждый сценарий — урок. slotclash показывает монополию делителя, wide_ilp — что ресурсы важнее пути.

АГЕНТ · ДИАЛОГ

АГЕНТ: отвечает только по числам, которые уже посчитаны

ШАГ 1

Вопрос

Обычным языком, в строке диалога

ШАГ 2

Досчитать

Если расписания ещё нет, агент считает его сам

ШАГ 3

Факты

Собирает список «что видит агент»: участок, числа, диагноз

ШАГ 4

Модель

Пересказывает только эти факты

ШАГ 5

Ответ и трасса

Показывает, что сделал, и команды для проверки

ПРАВИЛА ЧЕСТНОСТИ

Модель не считает и не видит исходник. Числа, которых нет в списке фактов, выдуманы. Каждый шаг агента можно повторить руками: `/doctor` `/compare` `/bounds`.

ПОДКЛЮЧЕНИЕ

Ключ API вставляется прямо в диалог и живёт только до закрытия терминала. Без ключа агент отвечает по уже посчитанным числам. Доступно 15 провайдеров и локальный сервер.

ПРИМЕР ВОПРОСОВ

› почему этот участок медленный?

› где теряются такты и что менять?

› загрузи examples/probe.s и разбери

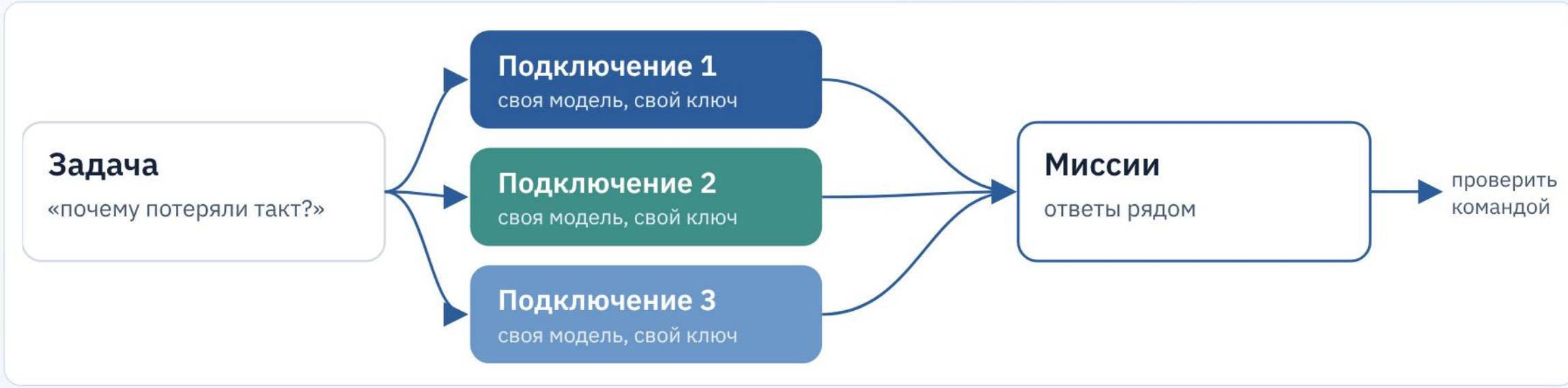
› что за монопольный порт ,5?

ПРИМЕР ТРАССЫ СПРАВА

```
✓ посчитал baseline
✓ посчитал оракул
✓ прочитал диагноз slotclash
→ проверь: /doctor
```

АГЕНТ · ШТАБ

ШТАБ: одна задача, несколько агентов в фоне



ЧТО ВЫБИРАЮ

Подключения, на которых пойдёт задача. Участок работы: потери, сравнение, предел, раскладка или свой вопрос. И скрипт-дописка к запросу.

ЧТО УМЕЮТ АГЕНТЫ

Только читать состояние. Не переключают сессию, не открывают файлы и ничего не пишут.

МИНИ И РАСКРЫТЫЙ ВИД

В мини хватает сводки и поля задачи, Enter отправляет. Двойной клик показывает подключения, скрипт и список миссий: ещё раз, убрать.

ЗАЧЕМ Чтобы сразу получить несколько мнений на одну и ту же задачу и сравнить их, не переключаясь между моделями.

ЯДРО · ИНТЕРПРЕТАТОР

ЯДРО: как граф получается из обычных вычислений

ЛЕНТА

› **a = 10**
→ 10

› **t = a*2**
→ 20 · граф: 1 оп.

› **sum 8**
→ готовое ядро, 15 оп.

› **go**
→ отдаю граф в РАЗБОР

ИМЕНА

```
a = 10
t = 20
```

ПРОГРАММА

```
load → a
mul a, 2 → t
store t
```

РАЗБОР

граф уже там

go: граф уходит в РАЗБОР

ЧТО ЭТО

Простой интерпретатор: считаю выражения, а он попутно строит граф зависимостей. По сути калькулятор с историей.

СЛОВА

`names` `mem` `list` `go` `reset` .
Готовые ядра: `sum 8` `dot 4` `saxpy 8` `fir 8`

ГДЕ ЖИВЁТ

Режим не удалён: клавиша 4 на экране выбора. Его мини-версия есть вкладкой «ядро» в КОДе.

ЧЕСТНО

Что гарантирую, а что нет

ЧТО Я ГАРАНТИРУЮ

• Расписание TINY AI законно по построению: его строит обычный код, а не сеть.

• TINY AI не хуже жадного: ничья отдаётся жадному.

• «Оптимум доказан» написано только там, где поиск действительно его доказал.

• Агент не считает сам. Каждый его шаг повторяется командой.

• Ключи API живут только в памяти сессии.

ЧЕГО Я НЕ ЗНАЮ ИЛИ НЕ ДОБИЛСЯ

• Такты модельные: на живом Эльбрусе не проверены.

• Аппаратная конвейеризация циклов (loop_mode) не моделируется.

• Языковые модели (Qwen, Gemma, Phi) расписания не улучшили.

• NNL пока фоновый и на настоящем коде почти не помогает.

• Запас на настоящем коде маленький: доли процента тактов.

подход Отрицательные результаты я не прячу: они лежат в репозитории рядом с хорошими, с методикой и скриптами проверки.

КАК ЗАПУСТИТЬ

Запуск за одну команду

```
# установка и запуск
pip install -e ".[tui]"
python3 -m vliw                    # экран выбора режимов
python3 -m vliw run slotclash      # сразу на примере

# массовые прогоны текстом
python3 -m vliw all                # все 17 сценариев
python3 -m vliw sweep              # случайные графы
python3 tools/tiny_fast_check.py # замеры TINY Fast
```

ТРЕБОВАНИЯ

Python 3.10+. Обязательных зависимостей нет. Для полноэкранных режимов нужны rich и textual.

ТЕСТЫ

668 тестов, 667 проходят. Один тест интерфейса (прокрутка решётки) у меня падает, причину ещё не разбирал.

ГДЕ ЧТО ЛЕЖИТ

Веса TINY AI в vliw/tiny, NNL в nnl_vliw и nex_core, документация в docs. Адаптеры Qwen опубликованы на Hugging Face.

ИТОГ

Что осталось в сухом остатке

1

Жадный уже хорош

На настоящем коде 94% блоков оптимальны, запас около 0,5% тактов.

2

TINY AI забирает почти весь запас

Возвращает 70% потерь за 0,6 мс и не ломает правила.

3

Языковая модель и NNL пока не помогли

Это записано и измерено. NNL остаётся фоном.

4

Главная ценность — микроскоп

Показывает, где и почему теряется такт, и проверяет это точным поиском.